Istifsar AI

Build Blueprint & Commit Guide

A history exploration platform where every AI answer is grounded in verified primary sources. No document, no history.

How to Use This Document

Each phase lists its tasks in build order, the dependencies to install (if any), and the exact git commit message to use when the task is done. Follow the phases sequentially — each phase depends on the one before it. Commit after each logical unit of work, not after each file change.

Commit convention: `type(scope): description`
Types: `feat`, `fix`, `chore`, `refactor`, `docs`
Scopes: `db`, `auth`, `ui`, `ai`, `ingest`, `rag`, `graph`, `discovery`, `contention`, `essay`, `config`

Phases Overview

0	**Foundation**	Scaffold, Supabase schema, auth, config
1	**Ingestion Pipeline**	Upload form, Edge Function, parsing, chunking, embedding, validation
2	**Query Core**	Hybrid retrieval, reranking, similarity gate, streaming chat, Split-Pane
3	**Modes & Conversations**	Raw/Interpreted toggle, Lens selector, conversation history
4	**Discovery Layer**	Homepage, browse surface, Rabbit Hole Engine, Archive Gaps
5	**Contention System**	Auto-detection, Contention Card, community vote
6	**Citation Graph**	React Flow graph, lazy loading, filter sidebar
7	**Living Essays & Profiles**	TipTap editor, citation mapper, historian profiles, Curated Paths
8	**Polish & Gamification**	Badges, streaks, sepia theme, mobile audit

Phase 0 — Foundation

Blocking dependencies. Nothing else runs without these. This phase takes the bare `create-next-app` scaffold and turns it into a project with a database, auth, and config layer.

Dependencies to Install

Package	Purpose
`@supabase/supabase-js`	Supabase client SDK
`@supabase/ssr`	Server-side Supabase helpers for Next.js
`shadcn/ui (via CLI)`	Component library (npx shadcn@latest init)

Tasks & Commits

#	Task	Git Commit
1	Initialize shadcn/ui, set up `components.json`, base theme	`chore(ui): init shadcn/ui`
2	Create `.env.example` with all required variable keys (Supabase, Gemini, Unstructured, Upstash, Resend)	`chore(config): add .env.example`
3	Create `lib/config/models.ts` — model name constants (fast, deep, embedding)	`feat(config): add model constants`
4	Create `lib/config/constants.ts` — similarity threshold (0.65), chunk size (600), overlap (100), TTLs	`feat(config): add pipeline constants`
5	Initialize Supabase (`npx supabase init`), write `0001_schema.sql` — all 17 tables	`feat(db): add schema migration`
6	Write `0002_rls.sql` — all RLS policies for tier-based access	`feat(db): add RLS policies`
7	Write `0003_indexes.sql` — pgvector HNSW + FTS tsvector indexes	`feat(db): add vector and FTS indexes`
8	Set up Supabase Auth + custom JWT claim hook (tier in `app_metadata`)	`feat(auth): add JWT claim hook`
9	Create Supabase Storage bucket (private, for document scans)	`feat(db): add storage buckets`
10	Create `lib/supabase/client.ts`, `server.ts`, `admin.ts`	`feat(auth): add supabase client helpers`

11	Seed `approved_institutions` table	`chore(db): seed approved institutions`
12	Implement Tier 1 auto-approval logic (email domain check)	`feat(auth): add tier-1 auto-approval`
13	Restructure `app/` into `(auth)/` and `(main)/` route groups	`refactor(ui): add route groups`
14	Create `proxy.ts` for route protection (auth redirect + tier gating)	`feat(auth): add proxy route protection`
15	Add `typecheck` script to `package.json`, verify build passes	`chore(config): add typecheck script`

Milestone: At the end of Phase 0, you have a Next.js app with Supabase connected, all tables created with RLS, auth working with tier-based JWT claims, and the config layer in place. No UI beyond the scaffold. Push to `main` and tag `v0.1.0-foundation`.

Phase 1 — Ingestion Pipeline

The platform has no value without documents. Build the full path from upload form to published document before touching any query UI.

Dependencies to Install

Package	Purpose
`@google/generative-ai`	Gemini SDK (embedding + auto-tagging)
`langchain / @langchain/textsplitters`	RecursiveCharacterTextSplitter for chunking
`resend`	Email notifications to validators
`unstructured-client`	Unstructured.io API for PDF parsing

Tasks & Commits

#	Task	Git Commit
1	Build `/contribute/upload` page — Source Dossier form (metadata + dual file upload: scan + transcription)	`feat(ingest): add upload form`
2	Create Supabase Edge Function (orchestrator): receives DB webhook on `documents` INSERT	`feat(ingest): add edge function scaffold`
3	Implement text extraction stage — use historian transcription if provided, else Unstructured.io API for scans	`feat(ingest): add text extraction`
4	Implement chunking stage — `RecursiveCharacterTextSplitter` (600 tokens, 100 overlap)	`feat(ingest): add chunking`
5	Implement embedding stage — `gemini-embedding-001` → 3072-dim vectors → `document_chunks`	`feat(ingest): add embedding`
6	Implement FTS indexing — build `tsvector` on transcription + calendar event extraction	`feat(ingest): add FTS indexing`
7	Implement auto-tagging — Gemini Flash extracts era, locations, persons, events → `document_tags`	`feat(ingest): add auto-tagging`
8	Build `/contribute/validate` page — side-by-side scan vs. transcription, approve/reject/flag	`feat(ingest): add validation queue`
9	Wire Resend email notifications to Tier 1 validators on new submission	`feat(ingest): add validator notifications`

10	Implement Tier 1 approval flow — 2 approvals → `published` status	`feat(ingest): add approval flow`

Milestone: A Tier 2 contributor can upload a document with metadata and scans. The Edge Function processes it (extract → chunk → embed → tag). Two Tier 1 validators can approve it. Tag `v0.2.0-ingestion`.

Phase 2 — Query Core

The core AI experience: ask a question, get a source-grounded answer with inline citations and a split-pane document viewer. Raw Evidence Mode only (Interpreted Mode comes in Phase 3).

Dependencies to Install

Package	Purpose
`ai`	Vercel AI SDK (streamText, useChat)
`@ai-sdk/google`	Gemini provider for Vercel AI SDK
`@upstash/redis`	Serverless Redis for response caching
`react-pdf / pdfjs-dist`	PDF.js viewer for Split-Pane

Tasks & Commits

#	Task	Git Commit
1	Implement `lib/ai/embed.ts` — query embedding via `gemini-embedding-001`	`feat(rag): add query embedding`
2	Implement `lib/ai/retrieve.ts` — hybrid retrieval (pgvector cosine + FTS tsvector + RRF merge)	`feat(rag): add hybrid retrieval`
3	Implement `lib/ai/rerank.ts` — Gemini Flash reranker (top 30 → top 8)	`feat(rag): add reranker`
4	Implement `lib/ai/gate.ts` — similarity gate (≥ 0.65) + `archive_gaps` upsert on miss	`feat(rag): add similarity gate`
5	Implement `lib/ai/prompts.ts` — Raw Evidence Mode system prompt (Agoncillo Constraint)	`feat(rag): add system prompts`
6	Build `app/api/chat/route.ts` — streaming chat endpoint via Vercel AI SDK `streamText`	`feat(rag): add chat API route`
7	Build `/ask` page — chat interface with `useChat` hook	`feat(ui): add ask page`
8	Build `CitationChip` component — parse `[DOC:chunk_id]` tokens inline in streamed response	`feat(ui): add citation chips`
9	Build `app/api/citations/[chunkId]/route.ts` — returns document metadata + signed URL	`feat(rag): add citation API`
10	Build `SplitPane` + `DocumentViewer` — PDF.js viewer (right pane), scroll to cited page on chip click	`feat(ui): add split-pane viewer`

11	Implement `lib/ai/cache.ts` — Upstash Redis get/set/invalidate by query hash	`feat(rag): add response caching`
12	Build `NoDocumentFallback` component — 'No document, no history' UI with Archive Gaps link	`feat(ui): add fallback component`

Milestone: A reader can type a question on `/ask`, receive a streamed answer with citation chips, click a chip to open the source document in the Split-Pane viewer, and see the 'No document, no history' fallback when evidence is insufficient. Tag `v0.3.0-query`.

Phase 3 — Modes & Conversations

Add Interpreted Mode (historian lens), mode switching UI, and persistent conversation history.

Tasks & Commits

#	Task	Git Commit
1	Build `ModeToggle` component — Raw Evidence / Interpreted switcher with confirmation dialog	`feat(ui): add mode toggle`
2	Add Interpreted Mode system prompt to `lib/ai/prompts.ts` (lens warning + essay abstract injection)	`feat(rag): add interpreted prompt`
3	Update `lib/ai/retrieve.ts` — mode filter: Layer 1 only (raw) vs Layer 1 + essay chunks at ×0.8 weight (interpreted)	`feat(rag): add mode-aware retrieval`
4	Build `LensSelector` component — essay picker modal for Interpreted Mode	`feat(ui): add lens selector`
5	Build `InterpretedModeBanner` — persistent amber/gold banner + warm chat pane tint	`feat(ui): add interpreted mode banner`
6	Implement conversation history — store/retrieve last 5 turns in `conversation_turns`	`feat(rag): add conversation history`
7	Implement mode switching logic — clears history, new conversation row, confirmation dialog required	`feat(rag): add mode switching`
8	Build conversation history sidebar on `/ask` page	`feat(ui): add conversation sidebar`

Milestone: A user can switch between Raw Evidence and Interpreted Mode. Selecting a Living Essay as a lens changes the retrieval scope, system prompt, and visual treatment of the entire chat pane. Tag `v0.4.0-modes`.

Phase 4 — Discovery Layer

The homepage and browse experience. The platform should reward wandering, not just querying.

Tasks & Commits

#	Task	Git Commit
1	Build `TodayInHistory` component — one document per day keyed to `calendar_events`	`feat(discovery): add today in history`
2	Build `FeaturedPath` component — admin-pinned curated path card	`feat(discovery): add featured path`
3	Build `ContentionSpotlight` component — most-voted open contention	`feat(discovery): add contention spotlight`
4	Build `GraphTeaser` component — animated citation graph preview	`feat(discovery): add graph teaser`
5	Assemble homepage (`app/(main)/page.tsx`) from discovery components	`feat(ui): build homepage`
6	Build `/explore` page — faceted filters (era, location, person, event, source_type) + document cards	`feat(discovery): add explore page`
7	Build `RabbitHolePanel` — related docs, citing essays, open contentions (shown on every document page)	`feat(discovery): add rabbit hole panel`
8	Build `/gaps` page — ranked list of unanswered queries by hit count + 'Contribute a source' CTA	`feat(discovery): add archive gaps page`

Milestone: The homepage drops users into specific content. The browse surface supports faceted search. Every document page has rabbit hole exits. Tag `v0.5.0-discovery`.

Phase 5 — Contention System

When primary sources contradict each other, the platform surfaces the conflict instead of hiding it. The AI never picks a side — it stops at the Contention Card.

Tasks & Commits

#	Task	Git Commit
1	Implement `lib/ingestion/contention.ts` — post-ingestion detection via Gemini Flash (top 5 similar chunks, binary contradiction check)	`feat(contention): add auto-detection`
2	Wire contention detection into the Edge Function ingestion pipeline (Stage 8)	`feat(contention): wire to ingestion`
3	Build `ContentionCard` component — inline in chat when conflicting sources are retrieved for the same query	`feat(contention): add contention card`
4	Build `/document/[id]/contention` page — full contention view with source excerpts	`feat(contention): add contention page`
5	Build `CommunityVote` component — non-binding signal vote (Source A vs B + optional reason)	`feat(contention): add community vote`
6	Implement Tier 1 resolution flow — cite resolving source → `status: resolved`	`feat(contention): add resolution flow`

Milestone: Contradictions between sources are auto-detected at ingestion. The chat surfaces Contention Cards. Readers can vote; Tier 1 can resolve. Tag `v0.6.0-contention`.

Phase 6 — Citation Graph

An interactive graph where documents and essays are nodes, citations are blue edges, and contentions are red edges. A navigation surface, not a verification overlay.

Dependencies to Install

Package	Purpose
`@xyflow/react`	React Flow for interactive graph rendering

Tasks & Commits

#	Task	Git Commit
1	Implement `lib/graph/build.ts` — fetch citation edges + contention edges, build node/edge data	`feat(graph): add graph data builder`
2	Build `CitationGraph` component — React Flow root with custom node/edge renderers	`feat(graph): add graph component`
3	Build `GraphNode` — size by in-degree, color by source_type, truncated title label	`feat(graph): add graph nodes`
4	Build `GraphEdge` — blue for citations, red for contentions	`feat(graph): add graph edges`
5	Build `NodePreviewCard` — hover preview (title, date, author, excerpt)	`feat(graph): add node preview`
6	Build `GraphFilterSidebar` — filter by era, source_type, historian	`feat(graph): add filter sidebar`
7	Implement lazy loading — 50 most-cited nodes default, expand neighbors on click, 'Load all' toggle	`feat(graph): add lazy loading`
8	Build `/explore/graph` page — full-page graph view	`feat(graph): add graph page`

Milestone: The citation graph renders interactively. Users can hover, click, filter, and navigate to documents. Tag `v0.7.0-graph`.

Phase 7 — Living Essays & Profiles

Historians publish analytical essays that become selectable lenses in Interpreted Mode. Public profiles and Curated Paths round out the contributor experience.

Dependencies to Install

Package	Purpose
`@tiptap/react + extensions`	Rich text editor for Living Essays

Tasks & Commits

#	Task	Git Commit
1	Build `EssayEditor` component — TipTap rich text editor with toolbar	`feat(essay): add essay editor`
2	Build `CitationMapper` component — link essay claims to vault documents	`feat(essay): add citation mapper`
3	Build `/contribute/essay` page — editor + submission flow + preview mode	`feat(essay): add essay page`
4	Implement essay peer review — 1 Tier 1 approval → `published`	`feat(essay): add peer review`
5	Build `/profile/[id]` page — uploads, essays, endorsements, tier badge, contribution stats	`feat(ui): add profile page`
6	Build `/paths` page — Curated Paths index (grid of path cards)	`feat(discovery): add paths index`
7	Build `/paths/[slug]` page — timeline card sequence with historian commentary	`feat(discovery): add path detail page`

Milestone: Historians can write, cite, and publish Living Essays. Published essays appear in the Lens selector. Curated Paths provide guided tours. Tag `v0.8.0-essays`.

Phase 8 — Polish & Gamification

Final polish: engagement features, theming, and mobile responsiveness.

Tasks & Commits

#	Task	Git Commit
1	Implement Explorer Badges — reading milestones (documents viewed, paths completed)	`feat(ui): add explorer badges`
2	Implement contention voting stats + leaderboard	`feat(contention): add voting stats`
3	Implement streak tracking — consecutive days visiting the platform	`feat(ui): add streak tracking`
4	Implement sepia/history theme — warm archival palette as shadcn theme option	`feat(ui): add sepia theme`
5	Mobile layout audit — Split-Pane → bottom sheet, graph → simplified list, verify all pages	`fix(ui): mobile responsive audit`
6	Performance audit — bundle size, Core Web Vitals, lazy loading verification	`chore(config): performance audit`

Milestone: The platform is production-ready with engagement features, mobile support, and theme options. Tag `v1.0.0`. Ship it.

Commit Quick Reference

Use conventional commits throughout the project. Each commit should represent a single logical unit of work that leaves the project in a buildable state.

Type	When to Use	Example
`feat`	New feature or capability	`feat(rag): add hybrid retrieval`
`fix`	Bug fix	`fix(auth): handle expired JWT refresh`
`chore`	Tooling, deps, config (no app logic)	`chore(config): add typecheck script`
`refactor`	Code restructure (no behavior change)	`refactor(ui): extract citation chip`
`docs`	Documentation only	`docs: update CLAUDE.md phase 1`

When to Commit

✓ Commit	✗ Don't Commit
A task from the phase checklist is fully done and `typecheck` passes	Mid-task with broken imports or incomplete logic
A bug fix that resolves a specific issue	'WIP' or 'saving progress' — use `git stash` instead
A refactor that changes structure but not behavior (verified by build)	Multiple unrelated changes in one commit
A config/tooling change that affects the dev workflow	`.env.local` or any secret

Version Tags

Tag	After Phase	What's Working
`v0.1.0-foundation`	Phase 0	DB, auth, config, route groups, proxy
`v0.2.0-ingestion`	Phase 1	Upload → process → validate → publish
`v0.3.0-query`	Phase 2	Ask → retrieve → stream → cite → Split-Pane
`v0.4.0-modes`	Phase 3	Raw + Interpreted modes, conversation history
`v0.5.0-discovery`	Phase 4	Homepage, explore, rabbit holes, archive gaps
`v0.6.0-contention`	Phase 5	Auto-detection, contention cards, community vote

`v0.7.0-graph`	Phase 6	Interactive citation graph with lazy loading
`v0.8.0-essays`	Phase 7	Essay editor, profiles, curated paths
`v1.0.0`	Phase 8	Production-ready: themes, mobile, gamification